



Multithreading Problem Bank (Java)

Legend

★ = Core Placement Problem (Mandatory)

Final file in the Java Core curriculum — covers thread fundamentals through the Executor Framework and `java.util.concurrent` utilities.

Section 1: Thread Basics and Creation

- ★ Create a Thread by Extending the Thread Class
- ★ Create a Thread by Implementing the Runnable Interface
- ★ Create a Thread using a Lambda Expression (Runnable as Functional Interface)

Start Multiple Threads and Observe Execution Order

- ★ Thread Naming using `getName()/setName()`

Thread Priority using `setPriority()`

Daemon Threads Demonstration

Section 2: Thread Lifecycle and States

- ★ Thread Lifecycle States (New, Runnable, Running, Blocked, Waiting, Terminated)
- ★ Pause Thread Execution using `sleep()`
- ★ Wait for Thread Completion using `join()`

Check Thread State using `getState()`

Interrupt a Thread using `interrupt()`

Section 3: Thread Synchronization (Core Concept)

- ★ Race Condition Demonstration (Without Synchronization)
- ★ Synchronized Method to Prevent Race Condition
- ★ Synchronized Block for Fine-Grained Locking
- ★ Deadlock Demonstration and Avoidance
- ★ Reentrant Lock using `java.util.concurrent.locks.ReentrantLock`

Difference between synchronized and volatile

Section 4: Inter-Thread Communication

- ★ Producer-Consumer Problem using `wait()` and `notify()`
 - `notifyAll()` vs `notify()` Demonstration
 - Thread Communication using a Shared Object
- ★ Bounded Buffer Implementation using `wait()/notify()`



Section 5: Executor Framework and Thread Pools (Java-Specific)

- ★ **Create a Fixed Thread Pool using Executors.newFixedThreadPool()**

- ★ **Submit Tasks using ExecutorService.submit()**

Cached Thread Pool vs Fixed Thread Pool Comparison

- ★ **Retrieve Results using Future and get()**

Shut Down an ExecutorService Gracefully

Scheduled Task Execution using ScheduledExecutorService

Section 6: Concurrency Utilities (java.util.concurrent)

- ★ **CountDownLatch for Coordinating Multiple Threads**

CyclicBarrier for Synchronizing a Group of Threads

Semaphore for Limiting Concurrent Access

- ★ **AtomicInteger for Thread-Safe Counters**

CompletableFuture for Asynchronous Computation

Section 7: Concurrent Collections

- ★ **ConcurrentHashMap vs Synchronized HashMap**

CopyOnWriteArrayList Demonstration

- ★ **BlockingQueue for Producer-Consumer without Manual wait/notify**

Section 8: Classic Concurrency Problems

- ★ **Producer-Consumer Problem using BlockingQueue**

Dining Philosophers Problem (Conceptual Simulation)

- ★ **Even-Odd Number Printing using Two Threads Alternately**

- ★ **Thread-Safe Singleton Class Implementation**

Section 9: Real World Multithreading Applications

- ★ **Multi-Threaded File Downloader Simulation**

- ★ **Bank Account Concurrent Transaction Simulation (Thread Safety)**

Ticket Booking System with Concurrent Access Control

- ★ **Multi-Threaded Matrix Multiplication**

Logging System with Thread-Safe Writes